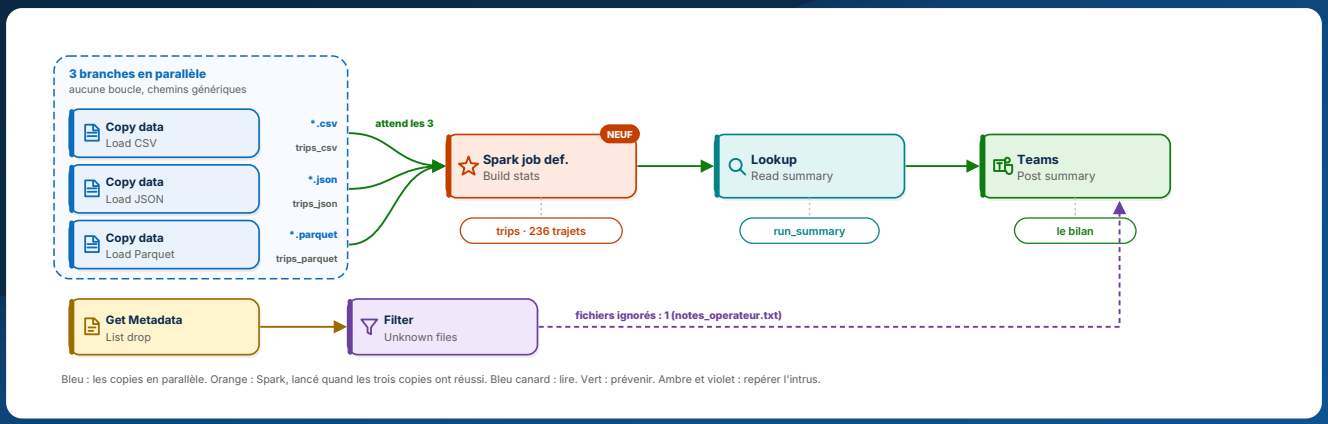
MICROSOFT FABRIC · DATA FACTORY · SPARK · GUIDE DE RÉFÉRENCE

Pipeline Microsoft Fabric : charger en parallèle, unifier, prévenir

CHARGER LISTER FILTRER UNIFIER LIRE PRÉVENIR

Trois partenaires de Lisbonne déposent leurs trajets de vélos, chacun dans son format : CSV, JSON, Parquet. Trois copies partent en même temps, sans boucle. Un programme Spark attend les trois, unifie les unités, écarte les doublons et désigne le quartier en tête. Un filtre repère l'intrus. Teams reçoit le bilan.



8	3	3	236	1
activités	formats	copies simultanées	trajets	intrus repéré

AUTEUR
Abdoul Hamid Diallo, Microsoft Data & AI Engineer

PRÉREQUIS
Un espace de travail Fabric sur une capacité, Microsoft Teams avec une équipe

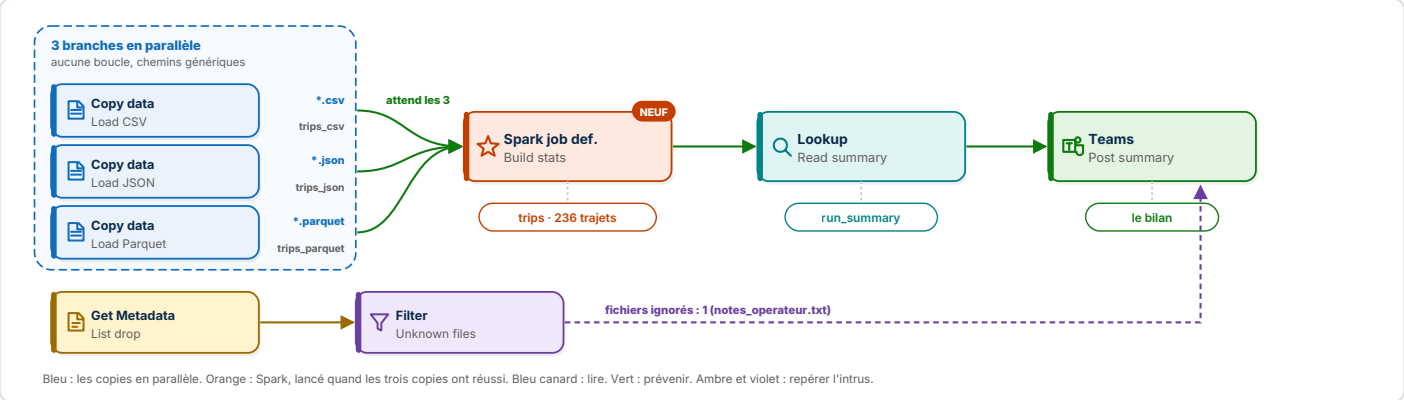
FICHIERS DU PROJET
Quatre fichiers de dépôt, le programme Spark lisboa_stats.py et expressions.txt

DURÉE
Environ 35 minutes, en quatre étapes vérifiées une à une

VUE D'ENSEMBLE

Ce que vous construisez

Lisbonne, lundi 19 octobre 2026. Trois partenaires exploitent les vélos en libre-service de trois quartiers et déposent chaque soir leurs trajets du jour. Chacun a ses habitudes : Belém envoie un **CSV** en minutes et kilomètres, Alfama un **JSON** en secondes et mètres, Parque das Nações un **Parquet** typé. Une note d'opérateur en **.txt** traîne dans le même dossier. Nouveautés : **la définition de tâche Spark**, **les branches parallèles qui se rejoignent**, **les chemins génériques**, **les formats JSON et Parquet**.



Le pipeline en une image : un bureau de tri

Trois guichets
Les copies reçoivent chacune un format, toutes en même temps.

Le portier
Get Metadata et Filter repèrent le colis qui n'a rien à faire là.

L'expert
Spark attend les trois guichets, met tout dans la même unité et élimine les doublons.

Le registre
Lookup lit la ligne de synthèse que l'expert a écrite.

Le haut-parleur
Teams annonce le bilan à l'équipe.

Les activités

ACTIVITÉ	TYPE	RÔLE
`Load CSV`, `Load JSON`, `Load Parquet`	Copy data PARALLÈLE	Trois copies sans lien entre elles : elles partent en même temps. Chemin générique `*.csv`, `*.json`, `*.parquet`
`List drop`	Get Metadata	Liste tout ce qui est dans `Files/drop`
`Unknown files`	Filter	Garde ce qui n'est ni CSV, ni JSON, ni Parquet : l'intrus
`Build stats`	Spark job definition NEUF	Attend les trois copies, puis lance `SJD_Lisboa` : unités unifiées, doublons écartés, verdict
`Read summary`	Lookup	Lit la ligne unique de `run_summary`
`Post summary`	Microsoft Teams	Attend Spark et le filtre, puis publie le bilan

FICHIER	PARTICULARITÉ	LIGNES
`belem_….csv`	Tout en texte, minutes, km	84
`alfama_….json`	Secondes, mètres, 4 trajets aussi déclarés par Belém	66
`parque_….parquet`	Typé, minutes, km	90
`notes_operateur.txt`	L'intrus	0

UN VRAI PROGRAMME SPARK La définition de tâche Spark exécute un fichier `.py` versionnable, comme en production, et non une cellule de notebook.

LE PARALLÈLE GRATUIT Pas de ForEach : des activités sans flèche entre elles s'exécutent ensemble. Une activité reliée à trois flèches vertes attend les trois.

ÉTAPE 1

Le lakehouse et le programme Spark

1 · OBJETS | 2 · CHARGER EN PARALLÈLE | 3 · REPÉRER L'INTRUS | 4 · UNIFIER ET PRÉVENIR

1 LH_Lisboa — LAKEHOUSE

1. Dans votre espace de travail, **Nouvel élément**, **Lakehouse**, nom `LH_Lisboa`.
2. Dans **Fichiers**, **Nouveau sous-dossier** : `drop`.
3. Ouvrez `drop`, **Charger**, **Charger des fichiers** : les **quatre** fichiers du dossier `data`, en une fois.

LE .TXT RESTE Il est là exprès : c'est lui que le filtre doit repérer.

2 SJD_Lisboa — DÉFINITION DE TÂCHE SPARK · NEUF

1. **Nouvel élément**, **Définition de tâche Spark** (Spark Job Definition), nom `SJD_Lisboa`. Langage : **PySpark (Python)**.
2. **Fichier de définition principal**, **Charger** : `lisboa_stats.py`.
3. **Référence Lakehouse**, **Ajouter** : `LH_Lisboa`, comme lakehouse par défaut.
4. **Enregistrer**. Ne l'exécutez pas encore : ses trois tables d'entrée n'existent pas.

```
# SJD_Lisboa : trois formats, trois unités, un seul jeu de trajets, puis le verdict par quartier
import sys
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName("SJD_Lisboa").getOrCreate()
run_date = sys.argv[1] if len(sys.argv) > 1 and sys.argv[1][:2] == "20" else "2026-10-19"  # marche aussi en notebook
elec = F.avg(F.when(F.col("bike_type") == "electrique", 1).otherwise(0)) * 100

def unify(table, duration, distance, to_min, to_km):  # tout en minutes et en kilomètres
    return spark.read.table(table).select(
        F.col("trip_id").cast("string").alias("trip_id"), F.col("station").cast("string").alias("station"),
        F.col("district").cast("string").alias("district"), F.to_timestamp(F.col("start_time").cast("string")).alias("start_time"),
        (F.col(duration).cast("double") / to_min).alias("duration_min"), (F.col(distance).cast("double") /
to_km).alias("distance_km"),
        F.col("bike_type").cast("string").alias("bike_type"))

raw = (unify("trips_csv", "duration_min", "distance_km", 1, 1)
       .unionByName(unify("trips_json", "duration_s", "distance_m", 60, 1000))
       .unionByName(unify("trips_parquet", "duration_min", "distance_km", 1, 1)))
rows_in = raw.count()
trips = raw.dropDuplicates(["trip_id"])
n = trips.count()
trips.write.mode("overwrite").option("overwriteSchema", "true").saveAsTable("trips")

stats = (trips.groupBy("district").agg(F.count("*").alias("trips"), F.round(F.avg("duration_min"), 1).alias("avg_duration_min"),
         F.round(F.sum("distance_km"), 1).alias("km_total"), F.round(elec, 1).alias("electric_pct")).orderBy(F.desc("trips")))
stats.write.mode("overwrite").option("overwriteSchema", "true").saveAsTable("district_stats")

top = stats.first()
tot = trips.agg(F.round(F.sum("distance_km"), 1).alias("km"), F.round(elec, 1).alias("pct")).first()
spark.createDataFrame(
    [(run_date, 3, rows_in, n, rows_in - n, top["district"], float(tot["km"]), float(tot["pct"]))],
    "run_date string, formats int, rows_in int, trips int, duplicates int, top_district string, km_total double, electric_pct
double",
).write.mode("overwrite").option("overwriteSchema", "true").saveAsTable("run_summary")
print(f"SJD_Lisboa : {rows_in} lignes lues, {n} trajets, {rows_in - n} doublons écartés")
```

unify()

Une fonction, trois appels : le JSON est divisé par 60 et par 1 000, le reste par 1. Tout sort en minutes et en kilomètres, typé.

dropDuplicates

240 lignes entrent. Les 4 trajets Belém vers Alfama, déclarés deux fois, n'en font plus qu'un chacun : 236.

run_summary

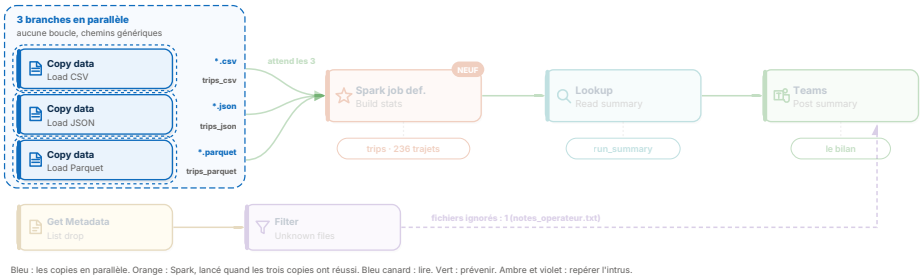
Une seule ligne, faite pour être lue par le pipeline : trajets, doublons, quartier en tête, km, part électrique.

AVANT LE PIPELINE ☐ quatre fichiers dans `drop` ☐ `lisboa_stats.py` chargé ☐ `LH_Lisboa` par défaut ☐ tout enregistré

ÉTAPE 2

Trois copies qui partent en même temps

1 · OBJETS | 2 · CHARGER EN PARALLÈLE | 3 · REPÉRER L'INTRUS | 4 · UNIFIER ET PRÉVENIR



Bleu : les copies en parallèle. Orange : Spark, lancé quand les trois copies ont réussi. Bleu canard : lire. Vert : prévenir. Ambre et violet : repérer l'intrus.

3 PL_Lisboa_Express

PIPELINE

1 **Nouvel élément**, **Pipeline**, nom `PL_Lisboa_Express`.

4 Les trois copies

COPY DATA · CHEMINS GÉNÉRIQUES

Copier les données, **Ajouter au canevas**, trois fois. Posez-les l'une sous l'autre et **ne les reliez pas** : c'est ce qui les fait partir ensemble.

Source, connexion	`LH_Lisboa`
Racine	`Fichiers`
Type de chemin	Chemin d'accès avec caractères génériques
Dossier	`drop`
Nom de fichier	le motif du tableau

Destination, connexion	`LH_Lisboa`
Racine	`Tables`
Table	Nouvelle, nom du tableau
Action de table	`Remplacer` (Overwrite)

NOM	MOTIF	FORMAT	PARTICULARITÉ DU FORMAT	TABLE
`Load CSV`	`*.csv`	**DelimitedText**	**Première ligne comme en-tête** coché	`trips_csv`
`Load JSON`	`*.json`	**JSON**	Rien à régler : un tableau d'objets, reconnu seul	`trips_json`
`Load Parquet`	`*.parquet`	**Parquet**	Rien à régler : le schéma est dans le fichier	`trips_parquet`

LE CHEMIN GÉNÉRIQUE `*.json` veut dire « tout fichier qui finit par .json ». Demain, le fichier du 20 octobre sera pris sans rien changer : aucun nom de fichier n'est écrit en dur.

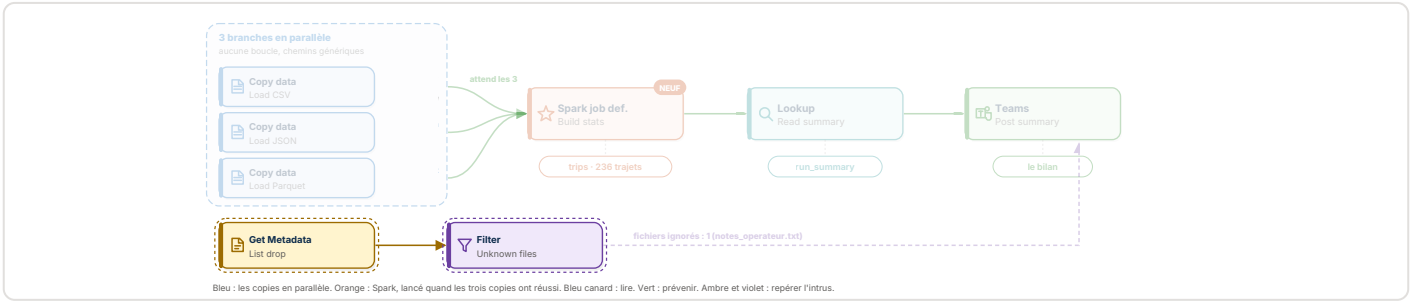
EXÉCUTION 1 **Enregistrer**, **Exécuter**. Les trois copies démarrent à la même seconde. Dans `LH_Lisboa`, **Tables**, **Actualiser** : `trips_csv` **84**, `trips_json` **66**, `trips_parquet` **90**.

TROIS TABLES, TROIS VÉRITÉS Ouvrez-les : `trips_csv` est tout en texte, `trips_json` en secondes et en mètres, `trips_parquet` typé. Impossible de les additionner telles quelles. C'est exactement le travail de Spark à l'étape 4.

ÉTAPE 3

Lister le dossier, filtrer l'intrus

1 · OBJETS | 2 · CHARGER EN PARALLÈLE | 3 · REPÉRER L'INTRUS | 4 · UNIFIER ET PRÉVENIR



5 List drop — GET METADATA

1. **Activités**, **Obtenir les métadonnées**, nom `List drop`. Posez-la sous les copies, sans la relier.
2. **Paramètres** : connexion `LH_Lisboa`, racine **Fichiers**, dossier `drop`, nom de fichier vide.
3. **Liste de champs**, **Nouveau** : **Éléments enfants**.

6 Unknown files — FILTER

1. **Filtre**, nom `Unknown files`, relié à la sortie verte de `List drop`.
2. **Éléments** :

```
@activity('List drop').output.childItems
```

3. **Condition** :

```
@not(or(or(endswith(toLower(item().name),
'.csv'), endswith(toLower(item().name),
'.json')), endswith(toLower(item().name),
'.parquet')))
```

CE QUE LIT LA CONDITION Pour chaque fichier, `toLower` met le nom en minuscules, `endswith` teste la fin du nom, `or` réunit les trois formats connus, `not` inverse : on garde tout ce qui n'est pas connu.

Exécution 2 — copies et filtre ensemble

Enregistrer, **Exécuter**. Cinq activités vertes. Sortie de `Unknown files` :

```
"FilteredItemsCount": 1, "Value": [ { "name":
"notes_operateur.txt", "type": "File" } ]
```

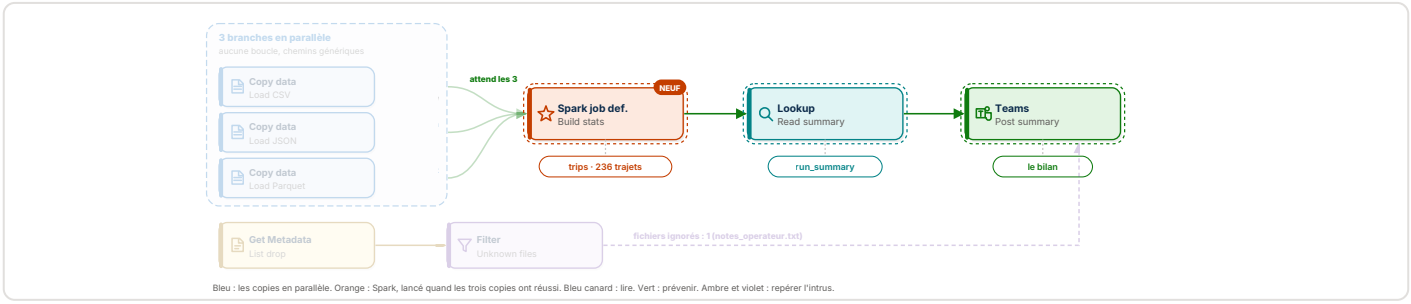
POURQUOI PAS UNE ERREUR Un fichier inattendu n'est pas une panne. Il est compté, et Teams le signalera à la fin. Le pipeline reste vert.

DEUX FAMILLES EN PARALLÈLE Les copies et le couple Get Metadata, Filter n'ont aucun lien : Fabric lance les quatre branches de départ en même temps. Regardez les heures de début dans la sortie.

ÉTAPE 4

Unifier avec Spark, lire le verdict, prévenir

1 · OBJETS | 2 · CHARGER EN PARALLÈLE | 3 · REPÉRER L'INTRUS | 4 · UNIFIER ET PRÉVENIR



7 Build stats

SPARK JOB DEFINITION · NEUF

1. **Activités**, **Définition de tâche Spark**, nom `Build stats`.
2. Reliez-y la sortie verte de **chacune des trois copies** : trois flèches arrivent sur `Build stats`.
3. **Paramètres** : votre espace de travail, élément `SJD_Lisboa`. Si une connexion est demandée : **Compte d'organisation**.

SI L'ACTIVITÉ N'EST PAS PROPOSÉE Créez un **Notebook** `NB_Lisboa`, lakehouse `LH_Lisboa`, collez `lisboa_stats.py` dans une cellule. Dans le pipeline, activité **Notebook** nommée `Build stats` à la même place. Même résultat.

8 Read summary

LOOKUP

1. **Exécutez une fois** pour que `run_summary` existe (deux à trois minutes, Spark démarre).
2. **Rechercher**, nom `Read summary`, derrière `Build stats`. Connexion `LH_Lisboa`, racine **Tables**, table `run_summary`. **Première ligne uniquement** coché.

9 Post summary

MICROSOFT TEAMS

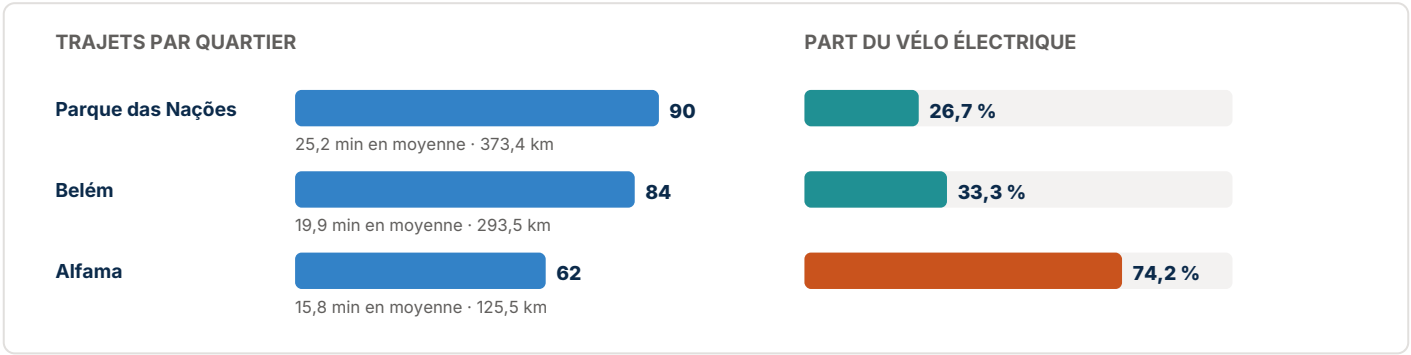
1. **Microsoft Teams**, nom `Post summary`. Reliez-y deux sorties vertes : `Read summary` et `Unknown files`.
2. Votre équipe, canal **Général**, **Titre** : `Vélos de Lisbonne`.
3. **Message** : la ligne **[Post summary]** de `expressions.txt`, en entier. **Enregistrer**, **Exécuter**.

Exécution 3 — le pipeline complet

Huit activités vertes. Dans Teams :



SOUS LE CAPOT Une activité reliée à plusieurs flèches vertes attend que toutes soient vertes. `Build stats` démarre donc à la fin de la plus lente des trois copies, jamais avant.



Trois réflexes d'expert sur ce pipeline

Un format de plus
Un partenaire passe en Avro ? Une quatrième copie `*.avro`, une ligne dans `unify()`, et rien d'autre ne bouge.

Le code à part
`lisboa_stats.py` se versionne dans Git et se remplace sans ouvrir le pipeline.

L'intrus compté
Un fichier inconnu n'arrête rien mais ne passe jamais inaperçu.

RÉFÉRENCE

Vérifier, retrouver une expression, débloquer

Vérifier dans le lakehouse

Ouvrez `LH_Lisboa`, **Point de terminaison SQL**, **Nouvelle requête SQL** :

```
SELECT district, trips, avg_duration_min,
       km_total, electric_pct
FROM district_stats
ORDER BY trips DESC;
```

DISTRICT	TRIPS	MIN	KM	% ÉLEC.
Parque das Nações	**90**	25.2	373.4	26.7
Belém	84	19.9	293.5	33.3
Alfama	62	15.8	125.5	**74.2**

240
lignes reçues en trois formats

4
doublons écartés par Spark

236
trajets uniques dans `trips`

74,2 %
d'électrique à Alfama, le quartier des collines

Les expressions

OÙ	EXPRESSION
Les trois copies, nom de fichier	`*.csv` · `*.json` · `*.parquet`
Unknown files, éléments	`@activity('List drop').output.childItems`
Unknown files, condition	`@not(or(or(endswith(toLower(item().name), '.csv'), endswith(toLower(item().name), '.json')), endswith(toLower(item().name), '.parquet')))`
Post summary, message	la ligne **[Post summary]** de `expressions.txt`

Si quelque chose bloque

SYMPTÔME	CAUSE ET CORRECTION
Une activité du guide est introuvable	Barre de recherche du ruban **Activités**. Pour **Définition de tâche Spark**, la variante Notebook de la page 6 donne le même résultat.
`Build stats` démarre avant la fin des copies	Une des trois flèches manque. Chaque copie doit avoir sa flèche verte vers `Build stats`.
`Build stats` : « Table or view not found: trips_csv »	Le lakehouse par défaut manque dans `SJD_Lisboa`. **Référence Lakehouse**, `LH_Lisboa`, **Enregistrer**.
Copie : « aucun fichier trouvé »	Le type de chemin est resté sur **Chemin d'accès au fichier**. Passez à **caractères génériques**, dossier `drop`.
`Read summary` ne propose pas `run_summary`	La table n'existe qu'après un passage de Spark. Exécutez une fois, puis **Actualiser**.
Échec « AADSTS700082 »	Connexion expirée. Recréez-la dans l'activité, **Compte d'organisation**, **Se connecter**.
Plus de 236 trajets	Une copie est en **Ajouter** au lieu de **Remplacer**. Corrigez, relancez : 236.